

# Formation FPGA

## TP2 – Compteurs

### 1) Calcul du nombre de périodes d'horloge pour atteindre 2 secondes

L'horloge système a une fréquence  $f_{clk}=100\text{ MHz}$ , ce qui correspond à une période

$$T_{clk} = \frac{1}{f_{clk}} = \frac{1}{100\text{ MHz}} = 10\text{ ns}.$$

Le nombre de périodes pour atteindre 2 secondes se calcule donc avec

$$N = \frac{2\text{ s}}{T_{clk}} = \frac{2\text{ s}}{10\text{ ns}} = 200\,000\,000.$$

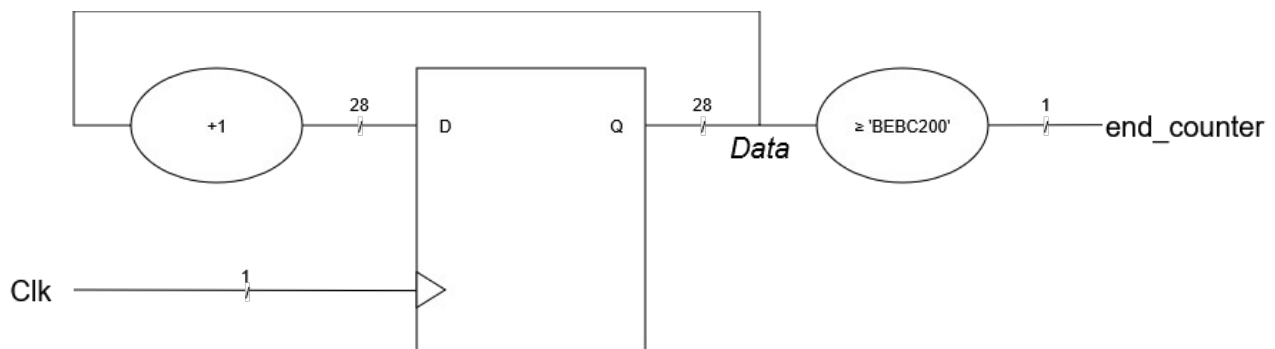
Pour calculer le nombre de bits nécessaires à l'encodage d'un compteur de période allant jusqu'à cette valeur N, on utilise un logarithme en base 2  $\log_2(N)=27,575$ .

En arrondissant cette valeur à l'entier supérieur, on trouve donc qu'il nous faut 28 bits pour encoder notre compteur (les 2 secondes sont atteintes quand le compteur atteint la valeur binaire '1011111010111100001000000000', soit une valeur hexadécimale 'BEBC200').

### 2) Schéma RTL du compteur « simple »

Il est constitué d'un incrémenteur (28 bits), d'un registre (28 bits) sans reset et d'un comparateur (28 bits).

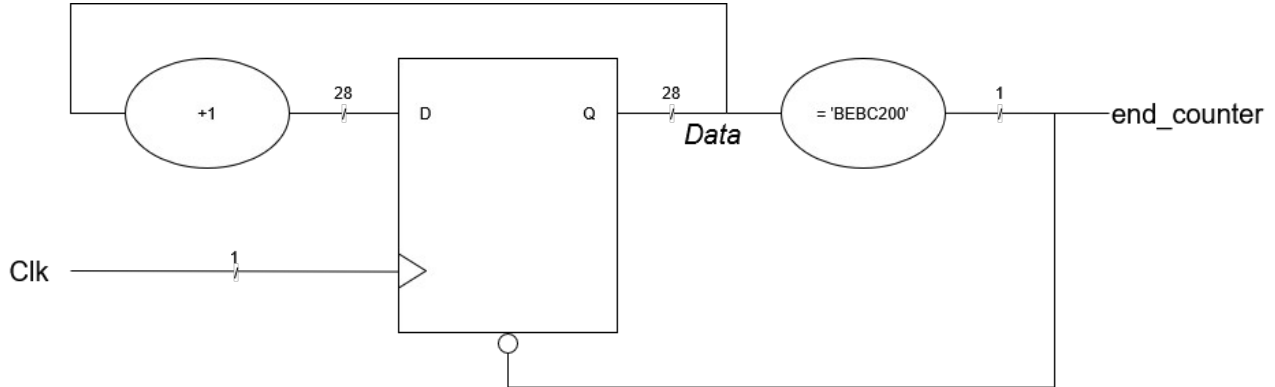
À chaque période d'horloge, la sortie du registre est incrémentée et réenregistrée. La sortie du registre est comparée à la valeur cible 'BEBC200' qui correspond à 2 s d'incrémentations, et qui déclenche le signal *end\_counter*.



### 3) Schéma RTL du compteur avec reset interne

Il est identique au précédent, mais cette fois le signal en sortie de la comparaison est également utilisé pour déclencher un reset du compteur (sur son état haut '1').

La comparaison peut être simplifiée en un simple '=' puisque dans cette configuration le compteur ne dépassera plus la valeur ciblée.



### 4) Signaux d'entrée, de sortie et internes

Entrée :

- Le signal d'horloge *clk* (1 bit)

Sortie :

- Le signal *end\_counter* (1 bit)

Internes :

- Le signal intermédiaire *Data* à la sortie du registre (28 bits)
- Le signal intermédiaire *reset\_cible* à la sortie du comparateur (1 bit), identique au signal de sortie *end\_counter*

### 5) Écriture du compteur en VHDL : voir fichier *counter.vhd*.

### 6) Écriture du testbench pour simulation : voir fichier *tb\_counter.vhd*.

Le seul signal externe de notre compteur étant son horloge, il n'y a aucun test particulier à faire sur des modifications de signaux d'entrée.

Le seul test effectué est de tester que le signal de sortie *end\_counter* vaut bien '1' au moment où on a atteint un cycle de compteur entier, qu'il repasse bien à '0' au cycle suivant, et finalement qu'il reprends la valeur '1' au bout de 2 cycles.

## 7) Simulation du compteur

Pour accélérer le calcul de la simulation, j'ai modifié le fichier *counter.vhd* le temps de celle-ci, pour définir la constante *cible* à 200000, soit un cycle de compteur de 2 ms.

On lance la simulation sur 5 ms, pour observer 2 cycles entiers du compteur.

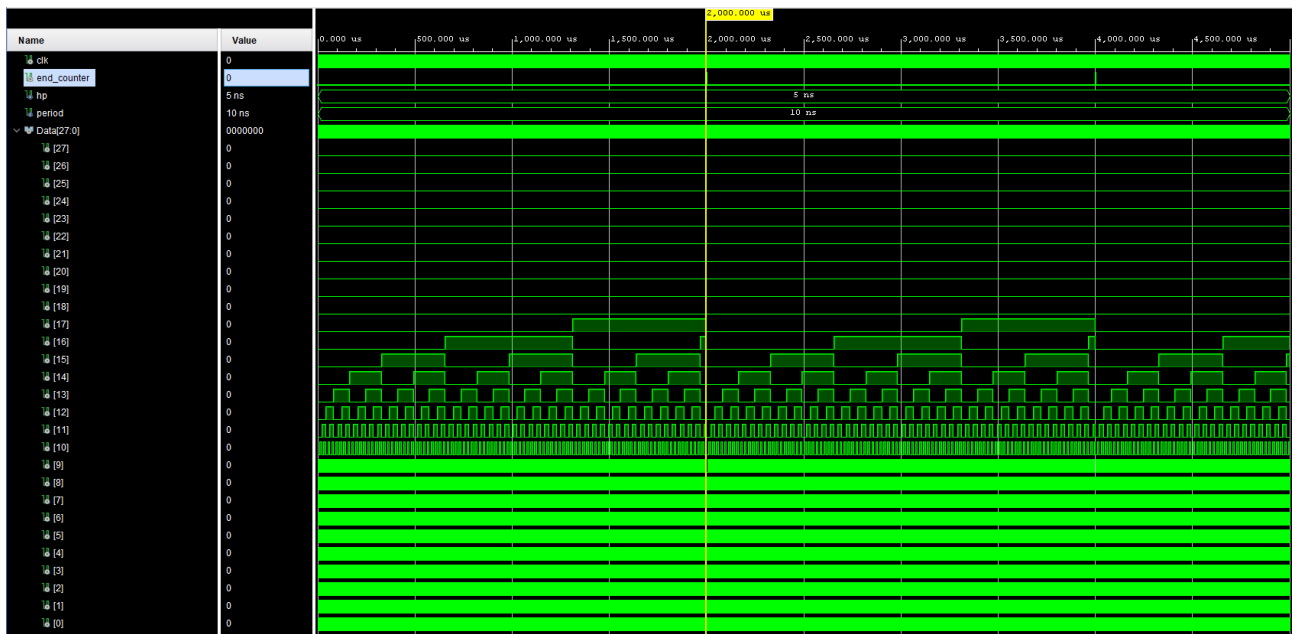
Sur le chronogramme, on doit observer le signal *end\_counter* faire un pic d'une période d'horloge à la fin du cycle de compteur, soit au bout de 2 ms, et un autre à 4 ms.

Le registre 28 bits *Data* doit se remplir en cascade au cour du cycle, le bit 0 oscillant tous les fronts montants de l'horloge, donc avec une période totale double de celle de l'horloge, les bits suivants voyant de même chacun leur période multipliée par deux par rapport au précédent.

La valeur de la durée totale du cycle ayant été divisée par 1000 pour la simulation, plusieurs des bits supérieurs ne seront jamais utilisés.

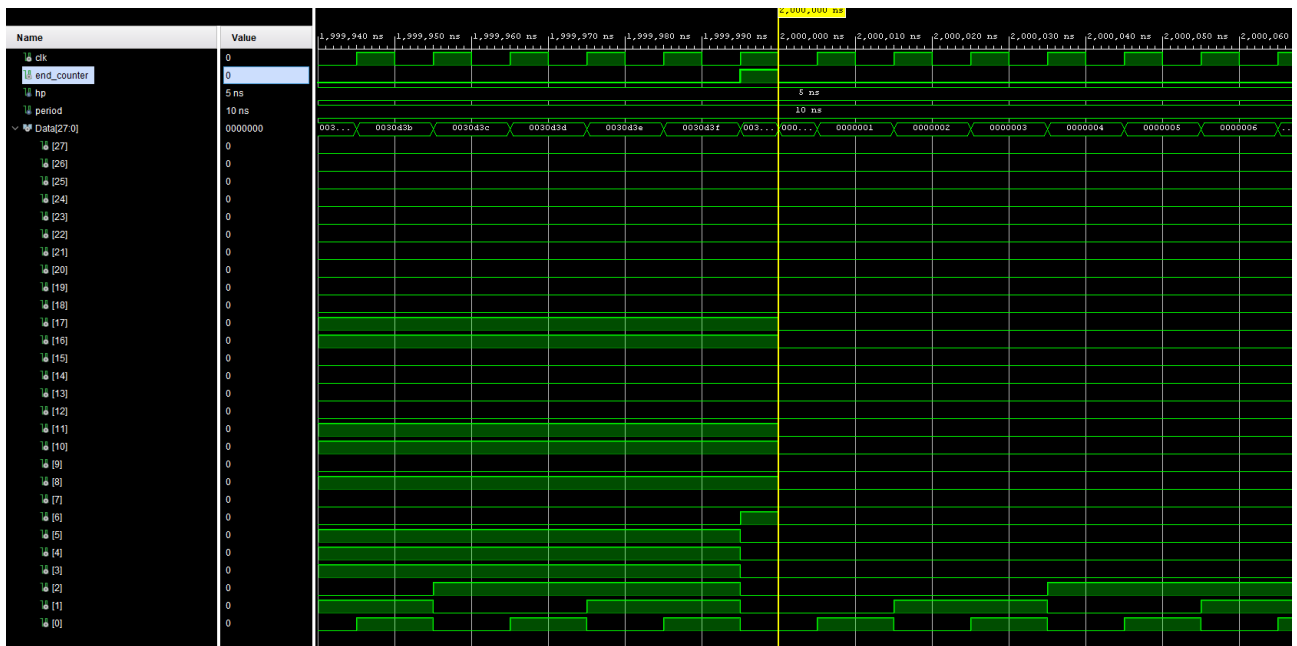
À la fin du cycle, quand on observe le pic de *end\_counter*, tous les bits de *Data* doivent être réinitialisés à 0.

Voici le chronogramme obtenu pour l'ensemble des 5 ms de simulations



On observe bien les pics de *end\_counter* à 2 ms et 4 ms, et le reset de *Data* à ces deux instants. *Data* se remplit bien en cascade pendant le cycle, et les bits 19 à 27 ne sont pas utilisés, comme on s'y attendait.

Voici un zoom du chronogramme à 2 ms, au moment du premier pic de *end\_counter* :



On observe que le pic de *end\_counter* commence 5 ns avant les 2 ms. C'est parce que l'horloge est initialisée à '0' à  $t=0$ , que son premier front montant à lieu après une demi-période à  $t=5$  ns, et donc que la première incrémentation de *Data* à lieu à ce même instant, et *Data* prends la valeur 1. Entre  $t=5$  ns et  $t=1\,999\,995$  ns, 199 999 cycle d'horloge supplémentaires ont eu lieu, *Data* s'est donc incrémenté 199 999 fois supplémentaires, et vaut la valeur cible 200 000. Le signal *end\_counter* prends donc l'état '1'.

Dans le fichier *counter.vhd*, le process (ie la partie séquentielle) est calé sur les changements d'états du signal d'horloge *clk*, il vérifie donc la condition de reset au front d'horloge suivant, le front bas à  $t=2$  ms. C'est à ce moment que le reset à lieu, que *Data* est réinitialisé à 0 et que le signal *end\_counter* repasse à l'état '0'. L'incrémentation reprend au front haut de l'horloge suivant, à  $t=2\,000\,005$  ns.

Ces résultats nous semblent satisfaisants, les tests intégrés au testbench *tb\_counter.vhd* sont validés :

```
Vivado Simulator 2020.2
Time resolution is 1 ns
Note: test end_counter = 1 at 2 ms OK
Time: 2 ms Iteration: 0 Process: /tb_counter/line__49
Note: test end_counter = 0 at 2.000010 ms OK
Time: 2000010 ns Iteration: 0 Process: /tb_counter/line__49
Note: test end_counter = 1 at 4 ms OK
Time: 4 ms Iteration: 0 Process: /tb_counter/line__49
```

## 8) Ajout d'une sortie LED.

On ajoute la sortie *led* (1 bit) au fichier source *counter.vhd*. Pour l'instant, elle possède la même valeur que la sortie *end\_counter*, et comme celle-ci sera haute pendant un seul coup d'horloge en fin de cycle.

On modifie le fichier de contraintes *Cora-Z7-10-Master.xdc* en conséquence pour relier cette sortie à la broche de la LED bleue 1.

9) On ajoute un signal de reset externe *restart* (1 bit), et on doit gérer la bascule de la LED entre son état allumé et éteint quand le compteur atteint sa cible.

Pour la gestion du reset, on a à la fois le nouveau signal externe *restart*, dont on voudrait que l'action soit asynchrone, et le signal interne déjà existant *r\_cible*, pour laquelle l'action doit être synchrone.

On va donc réorganiser la logique RTL autour du registre compteur. Son entrée reset sera pilotée de manière asynchrone par le signal *restart*, la réinitialisation étant déclenchée par son état haut '1'. La réinitialisation par le signal *r\_cible* ne sera plus gérée par l'entrée reset, mais au niveau de l'entrée de données D. En amont de celle-ci on va positionner un multiplexeur, commandé par le signal *r\_cible* : lorsque *r\_cible* vaut '0' on garde le fonctionnement normal d'incrémentation de *Data*, lorsque *r\_cible* vaut '1' on remet les 27 premiers bits de *Data* à '0' et le dernier à '1' (puisque cette réinitialisation a lieu en synchrone, c'est comme si on avait réinitialisé à 0 puis directement incrémenté de 1 comme dans un fonctionnement normal).

Pour gérer l'état de la LED, on veut détecter le front montant du signal *r\_cible* (ie la fin d'un cycle de compteur) et utiliser cette information pour déclencher la bascule entre l'état allumé et éteint de la LED. Pour cela, on a besoin de deux nouveaux registres, les deux sont actionnés par l'horloge *clk* et soumis au signal de reset externe *restart*.

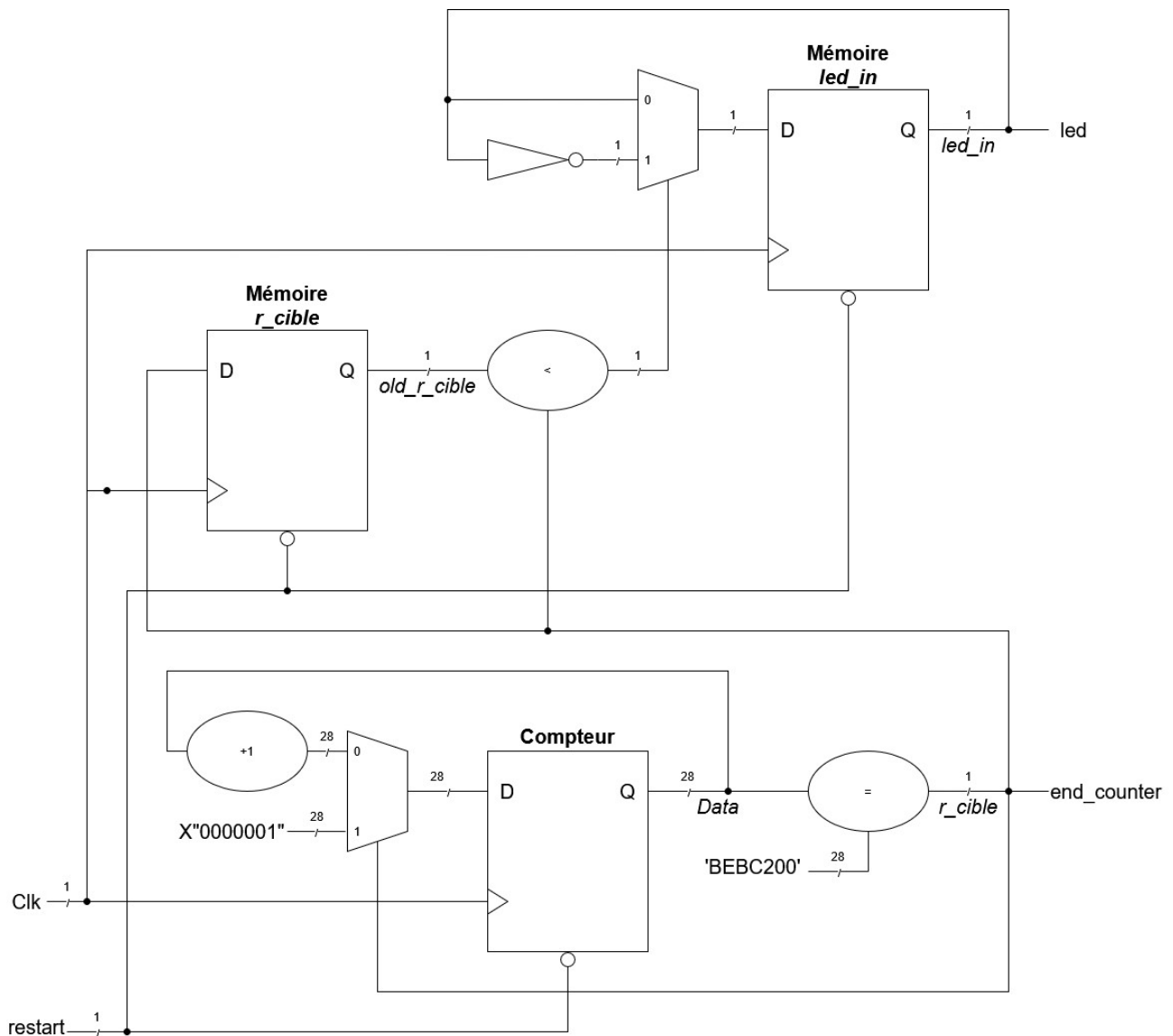
Le premier registre garde en mémoire la dernière valeur du signal *r\_cible*, et le stock sur un signal interne *old\_r\_cible*. Pour détecter le front montant du signal *r\_cible*, on compare les valeurs des deux signaux : au front montant on a  $old\_r\_cible = 0$  et  $r\_cible = 1$ , donc  $old\_r\_cible < r\_cible$ .

Le deuxième registre garde en mémoire la valeur de *led\_in*. On utilise un multiplexeur commandé par le signal issu de la comparaison précédente pour mettre à jour la valeur du registre :

- Lorsque la comparaison est fausse (le signal de commande vaut '0'), on n'est pas sur un front haut et donc la valeur *led\_in* reste inchangé.
- Lorsque la comparaison est vraie (le signal de commande vaut '1'), on est sur un front haut et on effectue la bascule de *led\_in* qui est remplacée par  $NOT(led\_in)$ .

La valeur de *led\_in* est ensuite redirigée vers la sortie *led*.

Voici le nouveau schéma RTL de notre compteur :



10) Mise à jour du code VHDL pour correspondre au nouveau schéma : Voir fichier *counter.vhd*.

Les lignes ajoutées sont signalées par un commentaire portant la mention « (question 10+) » en fin de ligne, les lignes supprimées par rapport à la version précédentes sont commentées, avec l'indication de la question pour laquelle la ligne était pertinente.

11) Association de l'entrée *restart* à un bouton : voir fichier de contraintes *Cora-Z7-10-Master.xdc*, où le signal est relié au bouton 1.

On profite de l'actualisation des contraintes pour :

- relier la sortie *end\_counter* à la LED 1 verte (et éviter que le signal ne soit lié à rien)
- créer le signal d'horloge système carré de période 10 ns et le relier à l'entrée *clk*

12) Mise à jour du testbench : voir fichier *tb\_counter.vhd*.

Les lignes ajoutées sont signalées par un commentaire portant la mention « (question 12+) » en fin de ligne.

On observe principalement :

- Les deux sorties du design :
  - *end\_counter* doit montrer un pic 2 ms après un reset, que celui ci soit interne lorsque le compteur atteint sa cible, ou externe lorsque le bouton lié a *restart* est appuyé.
  - *led* doit basculer d'état après chaque pic de *end\_counter*, et revenir à '0' lors d'un reset externe provoqué par le bouton *restart*.
- L'entrée *restart*, qui permet de vérifier le timing des sorties par rapport à ses changements d'états.

En pratique, le scénario de test mis en place dans notre testbench est le suivant (avec une valeur cible pour le compteur toujours raccourcie pour correspondre à un cycle de 2 ms) :

- 7 millisecondes de fonctionnement normal sans action externe.  
Toutes les 2 millisecondes, on vérifie que le signal *end\_counter* est à '1'.  
Au battement d'horloge suivant (10 ns plus tard), on vérifie que *end\_counter* est repassé à '0' et que *led* a basculé d'état ('1' à 2 ms, '0' à 4 ms, '1' à 6 ms).
- À  $t = 7$  ms, on appuie sur le bouton *restart*, pendant 0,5 ms.  
On vérifie qu'au premier battement d'horloge après le déclenchement du *restart*, la *led* s'éteint et repasse à l'état '0'.  
On vérifie ensuite que 2 ms après le relâchement du bouton, le signal *end\_counter* passe à '1'.  
Au battement d'horloge suivant on vérifie que *end\_counter* est repassé à '0' et que *led* s'allume en passant à l'état '1'.

Voici le chronogramme obtenu en simulation :



Visuellement, il correspond bien à ce qui était attendu.

Les tests réalisés dans le testbench sont validés :

```
Note: test end_counter = 1 at 2 ms OK
Time: 2 ms Iteration: 0 Process: /tb_counter
Note: test end_counter = 0 at 2.000010 ms OK
Time: 2000010 ns Iteration: 0 Process: /tb_c
Note: test led = 1 at 2.000010 ms OK
Time: 2000010 ns Iteration: 0 Process: /tb_c
Note: test end_counter = 1 at 4 ms OK
Time: 4 ms Iteration: 0 Process: /tb_counter
Note: test led = 0 at 4.000010 ms OK
Time: 4000010 ns Iteration: 0 Process: /tb_c
Note: test end_counter = 1 at 6 ms OK
Time: 6 ms Iteration: 0 Process: /tb_counter
Note: test led = 1 at 6.000010 ms OK
Time: 6000010 ns Iteration: 0 Process: /tb_c
Note: At 7 ms, restart is fixed as '1'
Time: 7 ms Iteration: 0 Process: /tb_counter
Note: test led = 0 at 7 ms OK
Time: 7000010 ns Iteration: 0 Process: /tb_c
Note: At 7.5 ms, restart is fixed as '0'
Time: 7500 us Iteration: 0 Process: /tb_coun
Note: test end_counter = 1 at 9.5 ms OK
Time: 9500 us Iteration: 0 Process: /tb_coun
Note: test end_counter = 0 at 9.500010 ms OK
Time: 9500010 ns Iteration: 0 Process: /tb_c
Note: test led = 1 at 9.500010 ms OK
Time: 9500010 ns Iteration: 0 Process: /tb_c
```



13) Voir schématique complète dans le fichier *Schématique\_TP2.png*

Les IBUF clk\_IBUF\_inst et restart\_IBUF\_inst sont les ports pour les deux signaux d'entrée de notre design (*clk* et *restart*).

Le BUFG clk\_IBUF\_BUFG\_inst est un buffer pour le signal d'horloge *clk*.

Les FDCE Data\_reg[0] à Data\_reg [27] sont des bascules, chacune enregistrant un des bits du signal *Data*, et constituent ensembles la partie stockage du registre « Compteur » sur le schéma RTL.

Les LUT2 Data[X]\_i\_x et les CARRY4 Data\_reg[X]\_i\_1 effectuent ensembles les opération (multiplexeur et incrémentation) avant la mise à jour des données du registre « Compteur », sur des séries de 4 bits.

Plus précisément, les LUT2 effectuent la réinitialisation des données lorsque la cible a été atteinte et le lancement de l'incrémentation pour le dernier bit, alors que les CARRY4 propagent cette incrémentation de bit en bit.

Les LUT4 et LUT6 end\_counter\_OBUF\_inst\_i\_x (x allant de 2 à 7) effectuent sur 4 ou 6 bits à la fois la comparaison du signal *Data* avec la cible X"BEBC200", avant de faire la comparaison final dans le LUT6 end\_counter\_OBUF\_inst\_i\_1 qui permet de créer le signal *r\_cible*.

Le FDCE old\_r\_cible\_reg est une bascule stockant le bit de donnée du registre « Mémoire *r\_cible* » du schéma RTL.

Le LUT3 led\_in\_i\_1 combine les actions du comparateur et du multiplexeur pour fournir la donnée mise à jour en entrée du registre « Mémoire *led\_in* » du schéma RTL.

Le FDCE led\_in\_reg est une bascule stockant le bit de donnée du registre « Mémoire *led\_in* » du schéma RTL.

Les OBUF clk\_IBUF\_BUFG\_inst et led\_OBUF\_inst sont les ports pour les deux signaux de sortie de notre design (*end\_counter* et *led*).

14) Voici les ressources utilisées d'après notre rapport de synthèse (voir aussi le rapport complet dans le fichier *counter\_unit.vds*) :

| Report Cell Usage: |        |       |
|--------------------|--------|-------|
|                    | Cell   | Count |
| 1                  | BUFG   | 1     |
| 2                  | CARRY4 | 7     |
| 3                  | LUT2   | 29    |
| 4                  | LUT3   | 1     |
| 5                  | LUT4   | 4     |
| 6                  | LUT6   | 3     |
| 7                  | FDCE   | 30    |
| 8                  | IBUF   | 2     |
| 9                  | OBUF   | 2     |

Par rapport à notre schéma RTL :

- Notre schéma comporte deux entrées, dont une horloge → cela correspond aux ressources d'entrée annoncées, **2 entrées IBUF** et **1 horloge BUFG**.
- Notre schéma comporte deux sorties → cela correspond aux ressources de sortie annoncées, **2 OBUF**.
- Notre schéma contient 3 registres, de respectivement 28 bits, 1 bit et 1 bit, pour un total de 30 bits → cela correspond aux ressources de mémoires, les **30 FDCE** annoncés.
- Pour l'entrée des données du registre « Compteur », notre schéma RTL prévoit :
  - Une incrémentation qui nécessite d'inverser l'état du dernier bit, et propager si besoin (c'est à dire si la valeur initial du dernier bit était '1') la retenue bit-à-bit sur les 27 bits. Cela nécessite donc une table de vérité à une entrée (LUT1) pour inverser le dernier bit, et de propager une retenue sur 28 bits (soit utiliser 7 CARRY4).
  - Un multiplexeur avec une des entrée variable à 28 bits (sortie de l'incrémenteur), l'autre entrée fixe (à X"0000001"), et une commande sur 1 bit. Pour chaque bit, cela correspond à un table de vérité à 2 entrées (la valeur du bit en entrée et la valeur du bit de commande), soit 28 LUT2.

Selon cette description, nous avons besoin au total pour cette sous-partie de 1 LUT1, 28 LUT2 et 7 CARRY4.

On peut remarquer que ce cheminement logique « incrémenter OU réinitialiser à 1 » est équivalent à « (réinitialiser à 0 OU rien) PUIS incrémenter ».

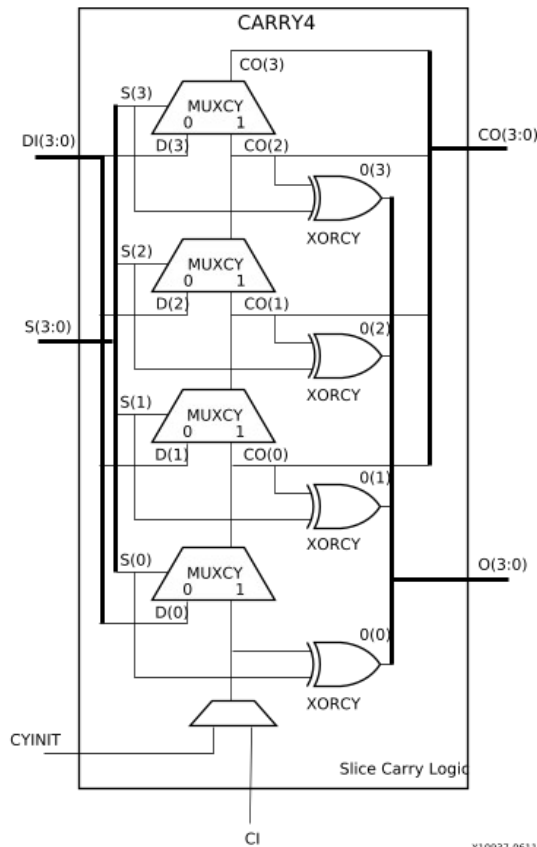
Cela revient à :

- Pour le dernier bit, éventuellement (si reset) le placer à '0', puis dans tous les cas l'inverser. Cela se traduit par une simple table de vérité à 2 entrées (LUT2)

| <i>r_cible</i> | <i>Data</i> [0] | <i>NewData</i> [0] |
|----------------|-----------------|--------------------|
| 0              | 0               | 1                  |
| 0              | 1               | 0                  |
| 1              | 0               | 1                  |
| 1              | 1               | 1                  |

Il faut aussi gérer le déclenchement de la retenue qui se propagera sur les bits supérieurs. À première vue, on veut que la retenue ne se déclenche que lorsque le dernier bit valait '1' et qu'il n'y a pas de reset, ce qui correspond à la table de vérité opposée à la précédente.

Cependant, en observant le fonctionnement exact de la primitive CARRY4, on remarque qu'une retenue indiquée fournie sur l'entrée D[0] aura un effet seulement si le bit d'entrée S[0] vaut '0'.



Cela veut dire que si on fournit sur S[0] le résultat de la table de vérité précédente (*NewData[0]*), l'entrée de retenue D[0] ne sera impliquée que dans le cas « *r\_cible*=0 et *Data[0]*=1 ». La valeur de D[0] n'a donc aucune importance lorsque « *r\_cible*=1 », et on peut piloter D[0] dans le cas « *r\_cible*=0 » en prenant directement la valeur de *Data[0]*, et en évitant une table de vérité supplémentaire.

- Pour les 27 autres bits, on applique l'éventuel reset à '0', puis on propage l'éventuel retenue venant du bit inférieur. Cela correspond à une LUT2, puis le passage dans un CARRY4.

Au total, cela demande une LUT2 par bit, soit 28 au total, et 7 CARRY4 (pour propager la retenue sur  $4 \times 7 = 28$  bits).

On observera qu'en pratique, le schéma de synthèse retient cette seconde méthode, avec 28 LUT2 en entrée des 7 CARRY4, mais en rajoutant une LUT2 supplémentaire pour piloter l'entrée de retenue sur le dernier bit, l'entrée D[0] sur le CARRY4 correspondant. Comme expliqué précédemment, je pense que cette LUT est superflue.

Au total, notre synthèse utilise donc **29 LUT2** et **7 CARRY4** pour ce sous-bloc.

- Sur notre schéma RTL, on obtient *r\_cible* en comparant les 28 bits de *Data* à une valeur fixe de 28 bits. Cette comparaison correspond donc à une table de vérité à 28 entrées.

Lors de la synthèse, obtenir directement une table de vérité à 28 entrées est impossible. En pratique, notre synthèse utilise pour cette opération dans un premier temps 2 LUT6 et 4 LUT4, qu'elle recombine ensuite dans une dernière LUT6. Cela représente au total **4 LUT4** et **3 LUT6**.

En revanche, je ne suis pas sûre de comprendre pourquoi la synthèse n'a pas économisé une LUT, en faisant la comparaison sur 4 LUT6 et 1 LUT4, puis la combinaison sur 1 LUT5, pour un total de 6 LUT (au lieu des 7 de la synthèse).

- Pour l'entrée du registre « Mémoire *led\_in* » du schéma RTL, les trois composants (comparaison, inverseur et multiplexeur) peuvent se résumer à une table de vérité à trois entrées : les signaux *r\_cible*, *old\_r\_cible* et *led\_in*.

Dans notre synthèse, cette logique est contenue dans **1 LUT3**.

Dans les ressources listées dans le rapport, ous retrouvons donc bien l'ensemble de la logique décrite dans notre schéma RTL.

15) Mise en place dans l'onglet « Set Up Debug » de Vivado de trois sondes ILA de debug, sur les nets :

- *end\_counter\_OBUF* (juste avant le port de sortie *end\_counter*) ;
- *led\_OBUF* (juste avant le port de sortie *led*) ;
- *restart\_IBUF* (juste après l'entrée *restart*)

16) Analyse du rapport de timing après implémentation (voir aussi le rapport complet dans le fichier *counter\_unit\_timing\_summary\_routed.rpt*).

Le récapitulatif de timing obtenu est le suivant :

| Design Timing Summary |         |                       |                     |         |         |                       |                     |          |          |                        |                      |
|-----------------------|---------|-----------------------|---------------------|---------|---------|-----------------------|---------------------|----------|----------|------------------------|----------------------|
| WNS(ns)               | TNS(ns) | TNS Failing Endpoints | TNS Total Endpoints | WHS(ns) | THS(ns) | THS Failing Endpoints | THS Total Endpoints | WPWS(ns) | TPWS(ns) | TPWS Failing Endpoints | TPWS Total Endpoints |
| 4.114                 | 0.000   | 0                     | 3638                | 0.047   | 0.000   | 0                     | 3622                | 3.750    | 0.000    | 0                      | 2018                 |

On observe notamment que les valeurs de *Total Negative Slack (TNS)* et de *Total Hold Slack (THS)* valent bien 0 ns, ce qui indique qu'il **n'y a ni violation de set up, ni violation de hold**.

Le récapitulatif d'horloge est le suivant :

```
-----  
| Clock Summary  
| -----  
-----
```

| Clock                                                                                      | Waveform(ns)   | Period(ns) | Frequency(MHz) |
|--------------------------------------------------------------------------------------------|----------------|------------|----------------|
| -----                                                                                      | -----          | -----      | -----          |
| dbg_hub/inst/BSCANID.u_xsdbm_id/SWITCH_N_EXT_BSCAN.bscan_inst/SERIES7_BSCAN.bscan_inst/TCK | {0.000 16.500} | 33.000     | 30.303         |
| sys_clk_pin                                                                                | {0.000 5.000}  | 10.000     | 100.000        |

L'horloge système *sys\_clk\_pin* est bien configurée selon nos spécifications initiales : période de 10 ns (soit une fréquence de 100 MHz), et signal carré (même demi-période de 5 ns pour les crêtes haute et basse du signal).

Le chemin critique présenté par le rapport est le suivant :

```
Max Delay Paths
```

```
-----  
Slack (MET) :      4.114ns (required time - arrival time)  
Source:          <hidden>  
                  (rising edge-triggered cell FDRE clocked by sys_clk_pin {rise@0.000ns fall@5.000ns period=10.000ns})  
Destination:     <hidden>  
                  (rising edge-triggered cell FDRE clocked by sys_clk_pin {rise@0.000ns fall@5.000ns period=10.000ns})  
Path Group:      sys_clk_pin  
Path Type:       Setup (Max at Slow Process Corner)  
Requirement:     10.000ns (sys_clk_pin rise@10.000ns - sys_clk_pin rise@0.000ns)  
Data Path Delay:  5.859ns (logic 1.266ns (21.606%) route 4.593ns (78.394%))  
Logic Levels:    4 (LUT2=1 LUT5=1 LUT6=2)  
Clock Path Skew: -0.072ns (DCD - SCD + CPR)  
Destination Clock Delay (DCD):  4.843ns = ( 14.843 - 10.000 )  
Source Clock Delay (SCD):        5.306ns  
Clock Pessimism Removal (CPR):   0.391ns  
Clock Uncertainty: 0.035ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE  
Total System Jitter (TSJ):       0.071ns  
Total Input Jitter (TIJ):        0.000ns  
Discrete Jitter (DJ):            0.000ns  
Phase Error (PE):               0.000ns
```

On remarque que le slack de ce chemin critique est à environ 40 % d'une période, ce qui est relativement large.

17) Le Bitstream est généré, et utilisé pour programmer la carte.

La vidéo *Démo\_compteur.mp4* présente 16 secondes de fonctionnement de la carte.

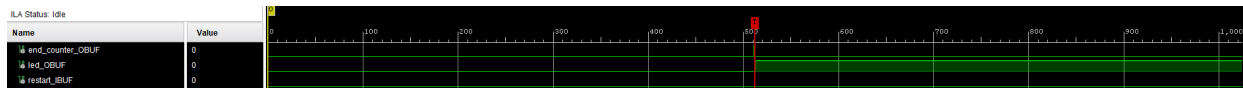
Les 6 premières secondes de vidéo présentent le fonctionnement normal de la carte, et on observe bien la LED clignoter (cycle de 2 secondes allumée, 2 secondes éteinte).

Autour de la 6ème seconde de vidéo, je presse le bouton *restart* alors que la LED est allumée : la LED s'éteint immédiatement et reste éteinte tant que le bouton est pressé.

Le bouton est relâché autour de la 9ème seconde de vidéo, et le fonctionnement normal reprends à partir de là jusqu'à la fin de la vidéo, en commençant par 2 secondes supplémentaires où la LED reste éteinte.

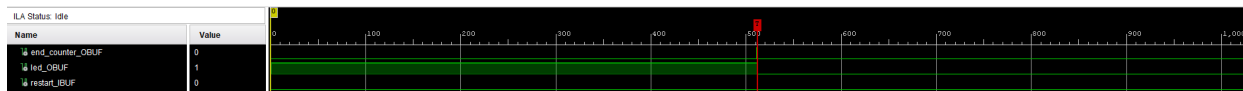
Plusieurs tests sont effectués avec la ILA :

- En mettant un trigger sur le front montant du signal *led*, on détecte en fonctionnement normal :



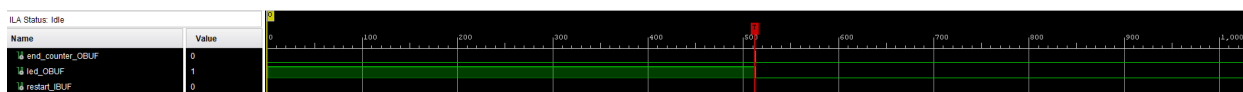
On observe bien juste avant le trigger le pic du signal *end\_counter*, qui indique que la bascule de la LED a été provoquée par une fin de compteur.

- De même, en mettant en fonctionnement normal un trigger sur le front descendant du signal *led*, on détecte :



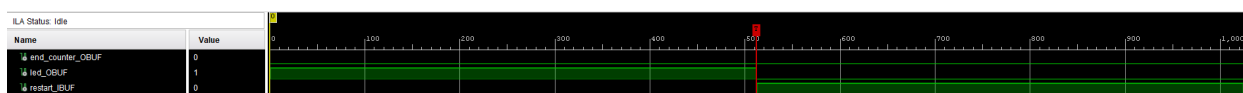
On observe encore une fois juste avant le trigger le pic du signal *end\_counter*.

- Inversement, en fonctionnement normal, en mettant un trigger sur un changement d'état du signal *end\_counter*, on détecte :



Dans ce cas, le pic de *end\_counter* a provoqué un passage de *led* du haut vers le bas.

- Enfin, en plaçant un trigger sur le front montant du signal *restart*, puis en appuyant sur le bouton alors que la LED est allumée, on obtient :



On observe bien le signal *restart* provoquer la mise à '0' du signal *led*, sans voir de pic sur *end\_counter*.

Que ce soit en visuel ou via l'ILA, la carte semble fonctionner parfaitement.